institut montilivi

TEORIA

REPÀS LINQ

MÒDUL

ACCÉS A DADES

Suposant la col·lecció base:

CODI C#

```
List<string> noms = new() { "Xevi", "Ferran", "Artur", "Jordi", "Narcis", "Elvis", "Nuria"
};
```

NÚM.	OPERACIÓ	DESCRIPCIÓ	EXEMPLE	RESULTAT
1️⃣	**Where**	Filtra els elements segons una condició.	`noms.Where(n => n.StartsWith("N"))`	`["Narcis", "Nuria"]`
2️⃣	**OrderBy**	Ordena els elements alfabèticament.	`noms.OrderBy(n => n)`	`["Artur", "Elvis", "Ferran", "Jordi", "Narcis", "Nuria", "Xevi"]`
3️⃣	**OrderByDescending**	Ordena els elements en ordre invers.	`noms.OrderByDescending(n => n)`	`["Xevi", "Nuria", "Narcis", "Jordi", "Ferran", "Elvis", "Artur"]`
4️⃣	**Select**	Projecta o transforma cada element.	`noms.Select(n => n.ToUpper())`	`["XEVI", "FERRAN", "ARTUR", ...]`
5️⃣	**Distinct**	Elimina els elements duplicats.	`noms.Distinct()`	Sense duplicats
6️⃣	**Count**	Compta els elements totals o els que compleixen una condició.	`noms.Count(n => n.Contains("a"))`	`3`
7️⃣	**Any**	Comprova si algun element compleix la condició.	`noms.Any(n => n.Length > 4)`	`true`
8️⃣	**All**	Comprova si tots els elements compleixen la condició.	`noms.All(n => n.Length > 2)`	`true`
9️⃣	**Contains**	Comprova si existeix un valor determinat.	`noms.Contains("Xevi")`	`true`
🔟	**FirstOrDefault**	Retorna el primer element que compleix la condició, si n'hi ha cap.	`noms.FirstOrDefault(n => n.StartsWith("N"))`	`"Narcis"`
11	**Last**	Retorna l'últim element.	`noms.Last()`	`"Nuria"`
12	**Take**	Agafa els primers *n* elements.	`noms.Take(3)`	`["Xevi", "Ferran", "Artur"]`
13	**Skip**	Omet els primers *n* elements.	`noms.Skip(2)`	`["Artur", "Jordi", "Narcis", "Elvis", "Nuria"]`
14	**Concat**	Uneix dues col·leccions, incloent-hi els duplicats.	`noms.Concat(new[] { "Pau" })`	`["Xevi", "Ferran", ..., "Nuria", "Pau"]`
15	**Union**	Uneix dues col·leccions sense duplicats.	`noms.Union(new[] { "Xevi", "Pau" })`	`["Xevi", "Ferran", ..., "Nuria", "Pau"]`
16	**Intersect**	Retorna els elements comuns entre dues col·leccions.	`noms.Intersect(new[] { "Xevi", "Nuria" })`	`["Xevi", "Nuria"]`
17	**Except**	Retorna els elements d'una col·lecció que no són a l'altra.	`noms.Except(new[] { "Xevi", "Artur" })`	`["Ferran", "Jordi", "Narcis", "Elvis", "Nuria"]`

NÚM.	OPERACIÓ	DESCRIPCIÓ	EXEMPLE	RESULTAT
18	**GroupBy**	Agrupa els elements segons una clau, com ara la longitud.	`noms.GroupBy(n => n.Length)`	Agrupa per longitud
19	**ToArray / ToList**	Converteix la col·lecció en un array o una llista.	`noms.ToArray()`	`string[]` amb els mateixos valors
20	**ToDictionary**	Converteix la col·lecció en un diccionari de claus i valors.	`noms.ToDictionary (n => n, n => n.Length)`	`{"Xevi":4, "Ferran":6, ...}`
21	**Select amb índex**	Afegeix l'índex de posició a cada element.	`noms.Select((n, i) => $"{i + 1}. {n}")`	`["1. Xevi", "2. Ferran", ...]`
22	**Single**	Retorna un únic element que compleix la condició.	`noms.Single(n => n == "Xevi")`	`"Xevi"`
23	**ThenBy**	Afegeix un segon criteri d'ordenació.	`noms.OrderBy(n => n.Length).ThenBy( n => n)`	Ordena per longitud i després alfabèticament
24	**Reverse**	Inverteix l'ordre actual dels elements.	`noms.Reverse()`	`["Nuria", "Elvis", "Narcis", ...]`
25	**Join**	Combina dues col·leccions segons una clau comuna.	`noms.Join(altres, n => n, o => o, (n, o) => n)`	Retorna les coincidències entre les dues col·leccions

Nota: els resultats de `Where`, `OrderBy`, `Select` i altres operacions són consultes LINQ que s'executen quan s'enumeren. En canvi, `ToArray()` i `ToList()` materialitzen els resultats en una col·lecció.